

Multi-Agent Systems



30 Oct 2025 – Lecture 2 - Unification and Search

Learning outcomes

Unification

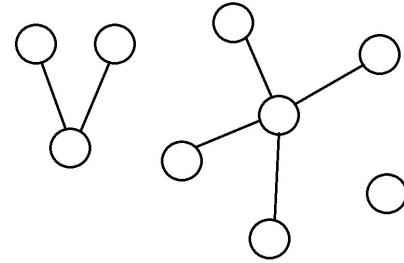
1. Student can explain how unification works in Prolog using the matching operator `=/2`.
2. Student can derive the result of a query that includes the matching operator.
3. Student can create new queries using the matching operator.

Search

1. Student can explain how search works in Prolog using backward chaining, linear traversal, and backtracking.
2. Student can create a search tree for a given program P and query Q.

How to learn - making the connection

- Many relations between topics taught
- Necessary for (deeper) understanding

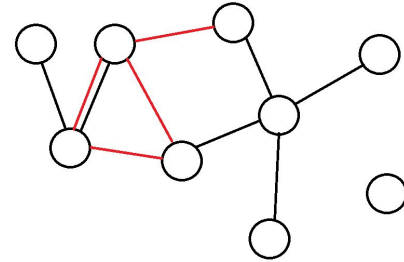


How to learn - making the connection

- Most valuable when you find them yourself
 - ask yourself questions
 - compare: how similar / how different

- We often don't do this across different courses

- Even within courses links important links aren't always put forward



How to learn - making the connection

What connections do you see with other courses?

Prolog - Unification

Multi-Agent Systems course

Lecture 2, part 1

?- life = life.
true.

Unification in Prolog

- Unification is solving a (logical) equation
- Matching is Prolog's method for **unification**
 - We generally still say unification when referring to Prolog's matching.
- We need to define when two predicates **match**

Unification in Prolog

Two terms t_1 and t_2 match ($t_1 = t_2$) iff:

1. t_1 and t_2 are identical constants.
 2. If t_1 is a (uninstantiated) variable and t_2 is not, then t_1 is instantiated with t_2
 - i. The same applies for the reverse or when both are variables.
 3. If t_1 and t_2 are compound terms, they match if:
 - i. They have the same *outermost functor*, i.e., $t_1 = f(r_1, \dots, r_k)$ and $t_2 = f(s_1, \dots, s_k)$
 - ii. Each term r_i matches with each term s_i
 - iii. The variable instantiations are *compatible*, i.e., a variable cannot be assigned different terms which are not variables.
- `=/2` is the built-in *matching predicate* (so not equality operator!)
 - Binds variables using the most general unifier
 - `\=/2` can be used to check for *terms that do not match*

?- ab = ab.
true.

?- =(ab,ab).
true.

?- apples \= oranges.
true.

?- $X = a.$

$X = a.$

?- $X = a, X = b.$

false.

?- $X \neq Y$.

false.

?- $X = f(a)$.

$X = f(a)$.

?- $f(X, g(a)) = f(b, g(Z))$.

$X = b, Z = a$.

?- $f(X,a) = f(b,X)$.

false.

?- $f(X,a) = f(b,Y)$.

$X = b$,

$Y = a$.

$$?- X+3 = 5+Y.$$

$$X = 5,$$

$$Y = 3.$$

?- $X+2 = 5$.

false.

?- term(X) = term(Y).

X = Y.

?- term(X,X) = term(Y,Z).
X=Y, Y=Z.

?- term(X,X) = term(X).
false.

Prolog - Search

Multi-Agent Systems course

Lecture 2, part 2

Deduction

Suppose we are given the following program and ask the following query:

```
q(a).  
r(a).  
p(X) :- q(X), r(X).
```

```
?- p(a).
```

What happens?

- match $p(a)$ with $p(X)$
- start with deriving body of rule: $q(a), r(a)$
- both goals are facts in the knowledge base.

Searching for Proofs

Searching for a **proof of a query**, or an **answer to a query** is a multiple step process:

1. Ask a query (and split into subgoals)
2. Find appropriate facts or rules (for subgoal)
3. Prove body of rule
4. Continue until no alternatives remain

Which rule/fact to apply first?

Logic program is ordered, not 'just' a set of rules

Example: Prolog Proof Search

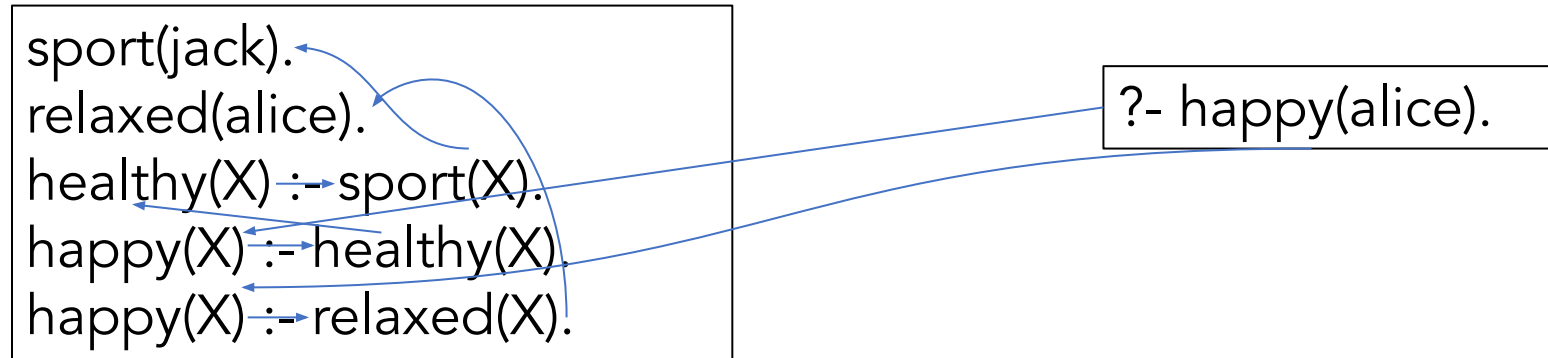
```
sports(jack).  
relaxed(alice).  
healthy(X) :- sports(X).  
happy(X) :- healthy(X).  
happy(X) :- relaxed(X).
```

```
?- happy(alice).
```

Three Principles

1. **backward chaining:** start from the current goal and select the rule/fact which allows to derive the goal. If a rule is matched, try to derive the rule's body (from left to right).
2. **linear:** traverse the logic program from top to bottom (select appropriate rules).
3. **backtracking:** if the current goal cannot further be matched with any rule, try other alternative rules in step 2 (i.e., the 2nd topmost, 3rd topmost, etc.). Backtrack to the closest possible choice point and try an alternative.

Example: Prolog Proof Search



Backward chaining compound queries



goal1(X) :- p1(t4), p2(t5).
...

?- goal1(t1), goal2(t2).

- Find the **first topmost** rule such that head of rule matches with goal1(t1) for an instantiation σ .
- Continue with new query (*body replacement*):
 - ?- p1(t4) σ , p2(t5) σ , goal2(t2) σ .
 - Note: (*sub*)goals are treated from left to right.

When can we stop?

query ?-

- ☐ No more goals to deduce (empty query)
- ☐ **success**

query ?- p(t), . . . and:

- No rule's head or fact matches with p(t)
 - ☐ What now? **failure**, backtrack to most recent choice point and try alternative rule.
 - ☐ No alternatives left: **terminate and fail**.

Standardisation

If a rule like

`head(X,Y) :- left(X), right(Y).`

is selected, Prolog:

- **introduces new variable names** and
- **replaces old ones uniformly.**

E.g., an instantiated rule looks as follows:

`head(_G4, _G453) :- left(_G4), right(_G453).`

Prolog search tree

Given a Prolog program P and a query Q the just described procedure results in a tree with the following properties:

- the **root** is labelled with query Q
- **nodes** are labelled with queries
- **child nodes** are ordered (according to the position of the rule in the program)
- **edges** are labelled with variable instantiations

We call this tree the **search tree of P and Q** (sometimes also called **proof tree**).

Search Tree Branches

A search tree can have 3 different types of branches:

- infinite ones,
- successful branches ending with the empty clause (and leading to an answer), and
- failing branches (dead ends).

Search tree exercise

Consider the following program and query:

```
s(a).  
r(b).  
p(X) :- r(X).  
p(X) :- s(X).  
p(X) :- t(X).  
t(X) :- t(X).
```

```
?- p(a).
```

Draw the search tree and identify for each branch whether it is succesful, failing, or inifite.

Search tree example

$p(a)$

Program

$s(a).$

$r(b).$

$p(X) \text{ :- } r(X).$

$p(X) \text{ :- } s(X).$

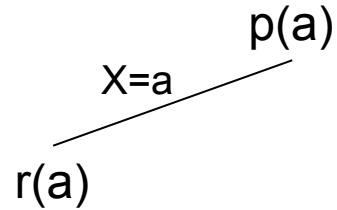
$p(X) \text{ :- } t(X).$

$t(X) \text{ :- } t(X).$

$?- p(a).$

Start: the root is labelled with the query.

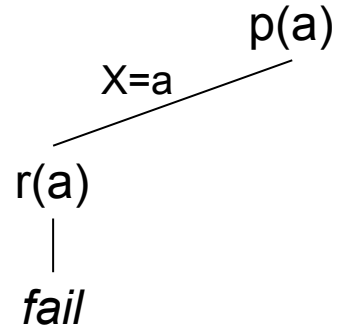
Search tree example

**Program**

```
s(a).  
r(b).  
p(X) :- r(X).  
p(X) :- s(X).  
p(X) :- t(X).  
t(X) :- t(X).  
  
?- p(a).
```

Match: first rule that matches query (no fact that matches).

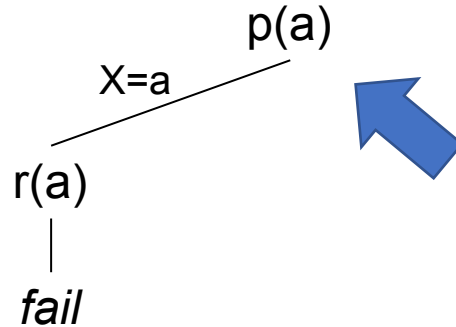
Search tree example

**Program**

```
s(a).  
r(b).  
p(X) :- r(X).  
p(X) :- s(X).  
p(X) :- t(X).  
t(X) :- t(X).  
  
?- p(a).
```

No alternative: failing branch.

Search tree example



Program

```

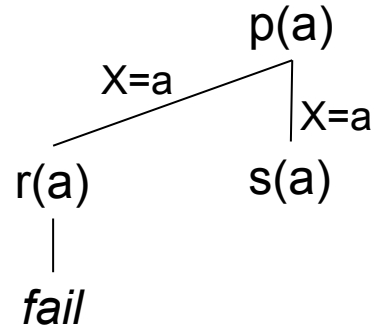
s(a).
r(b).
p(X) :- r(X).
p(X) :- s(X).
p(X) :- t(X).
t(X) :- t(X).

?- p(a).
  
```

~~striketrough~~=used

Backtrack: to node above failing node which has alternatives.

Search tree example



Program

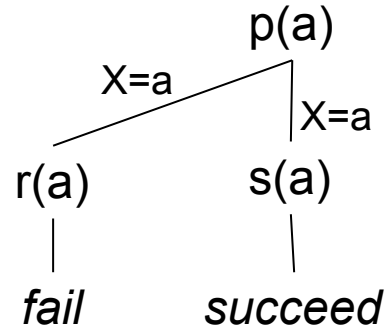
```

s(a).
r(b).
p(X) :- r(X).
p(X) :- s(X).
p(X) :- t(X).
t(X) :- t(X).

?- p(a).
  
```

Match: second rule matches query.

Search tree example



Program

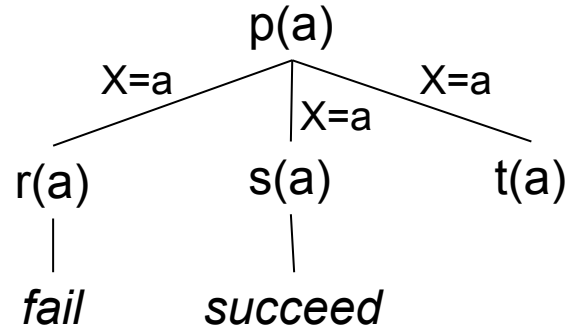
```

s(a).
r(b).
p(X) :- r(X).
p(X) :- s(X).
p(X) :- t(X).
t(X) :- t(X).

?- p(a).
  
```

Match with fact: success!

Search tree example



Program

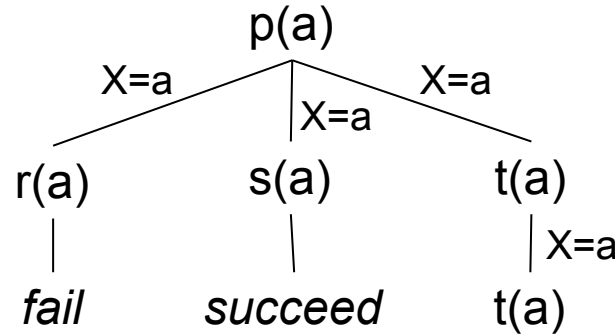
```

s(a).
r(b).
p(X) :- r(X).
p(X) :- s(X).
p(X) :- t(X).
t(X) :- t(X).

?- p(a).
  
```

Continue with third matching rule

Search tree example



Program

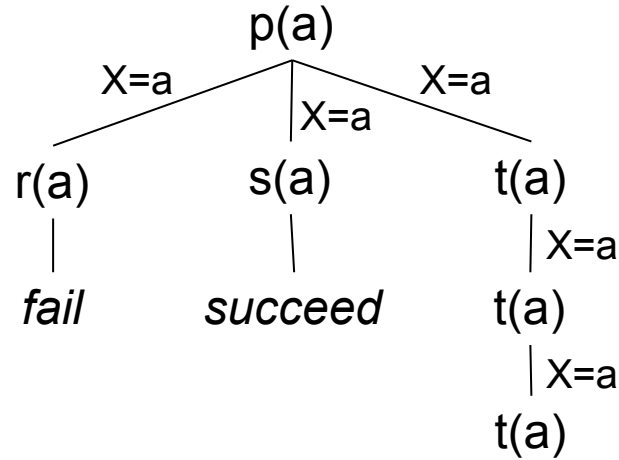
```

s(a).
r(b).
p(X) :- r(X).
p(X) :- s(X).
p(X) :- t(X).
t(X) :- t(X).

?- p(a).
  
```

Match with last rule

Search tree example



Program

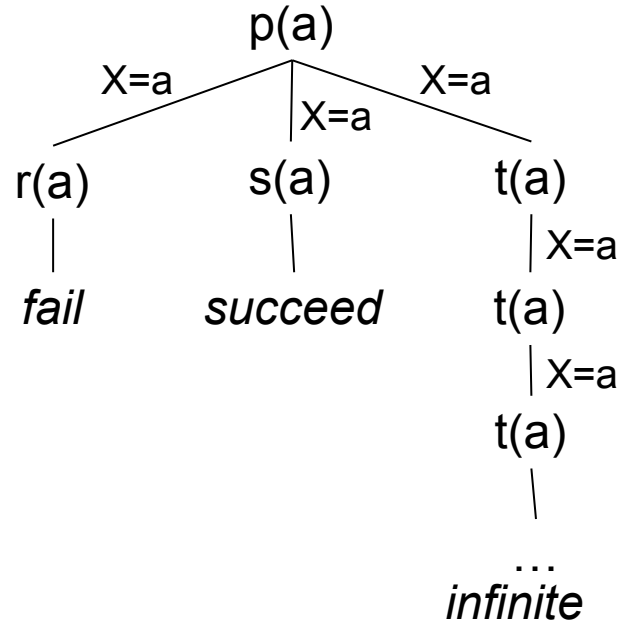
```

s(a).
r(b).
p(X) :- r(X).
p(X) :- s(X).
p(X) :- t(X).
t(X) :- t(X).

?- p(a).
  
```

Match with last rule

Search tree example



Program

```

s(a).
r(b).
p(X) :- r(X).
p(X) :- s(X).
p(X) :- t(X).
t(X) :- t(X).

?- p(a).
  
```

Only thing we can do is to match (again!) with last rule...

Search Tree – Variables

```
k(a).  
l(b,c).  
l(b,d).  
p(X, Y) :- k(X), l(Y, Z), not(Z=c).
```

```
?- p(a, Z).
```

Search Tree – Variables

$p(a, Z)$

Program

$k(a).$

$l(b,c).$

$l(b,d).$

$p(X, Y) \text{ :- } k(X), l(Y, Z), \text{not}(Z=c).$

$?- p(a, Z).$

Start: the root is labelled with the query.

Search Tree – Variables

$p(a, Z)$
| $X=a, Y=Z$
 $k(a), l(Z, Z), \text{not}(Z=c).$

Program

$k(a).$

$l(b,c).$

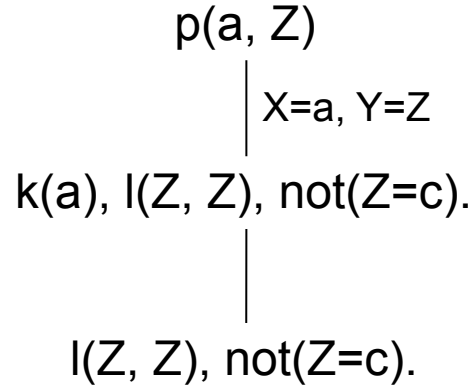
$l(b,d).$

$p(X, Y) \text{ :- } k(X), l(Y, Z), \text{not}(Z=c).$

$?- p(a, Z).$

Match with rule.

Search Tree – Variables



Program

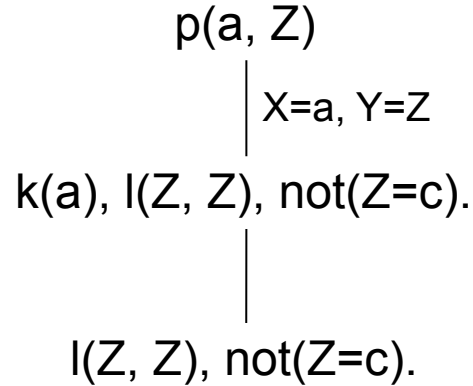
```

k(a).
l(b,c).
l(b,d).
p(X, Y) :- k(X), l(Y, Z), not(Z=c).

?- p(a, Z).
    
```

Match with fact k(a).

Search Tree – Variables



Program

```

k(a).
l(b,c).
l(b,d).
p(X, Y) :- k(X), l(Y, Z), not(Z=c).

?- p(a, Z).
    
```

Something has gone wrong...
Nothing to match $l(Z,Z)$ against...

But instantiation: $p(a,b) :- k(a), l(b,d), \text{not}(d=c)$ would yield answer $Z=b$ to our query $p(a, Z)$!

Search Tree – Variables

$$\begin{array}{c}
 p(a, Z) \\
 | \\
 X=a, Y=Z \\
 k(a), l(Z, Z), \text{not}(Z=c).
 \end{array}$$

not ok!

Program

```

k(a).
l(b,c).
l(b,d).
p(X, Y) :- k(X), l(Y, Z), not(Z=c).

?- p(a, Z).
    
```

Match with rule... but make sure you take care to
rename variables used in rule to avoid variable clashes!

Search Tree – Variables

$p(a, Z)$
 $\swarrow$ $_G23=a, _G24=Z$
 $k(a), l(Z, _G25), \text{not}(_G25=c).$

Program

```

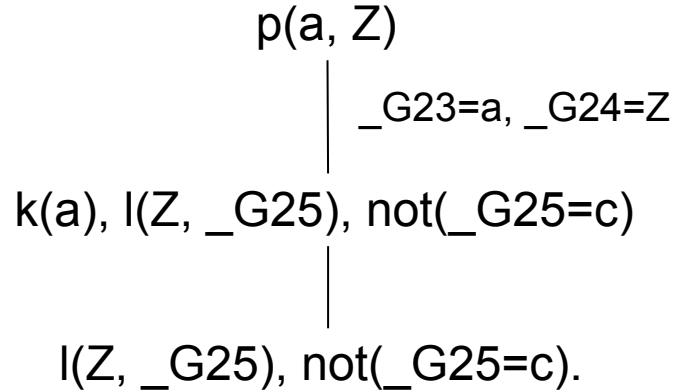
k(a).
l(b,c).
l(b,d).
p(X, Y) :- k(X), l(Y, Z), not(Z=c).

?- p(a, Z).
    
```

Match with rule but use instance with renamed variables:

$p(_G23, _G24) \text{ :- } k(_G23), l(_G24, _G25), \text{not}(_G25=c).$

Search Tree – Variables



Program

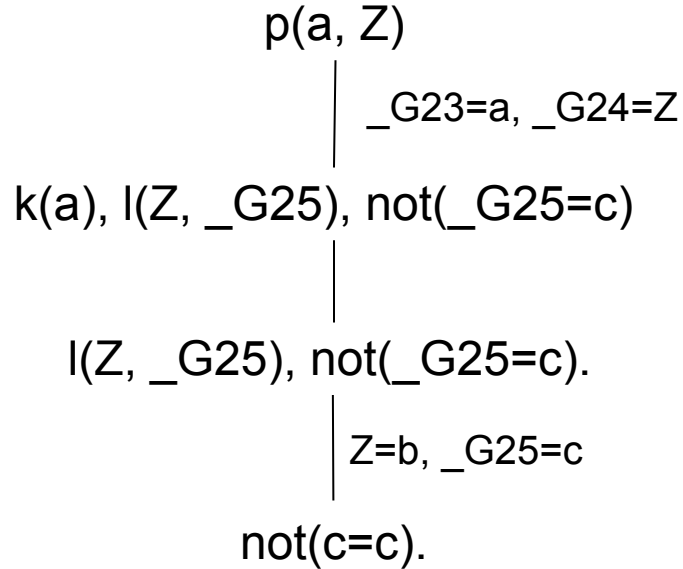
```

k(a).
l(b,c).
l(b,d).
p(X, Y) :- k(X), l(Y, Z), not(Z=c).

?- p(a, Z).
    
```

Match with first fact

Search Tree – Variables



Program

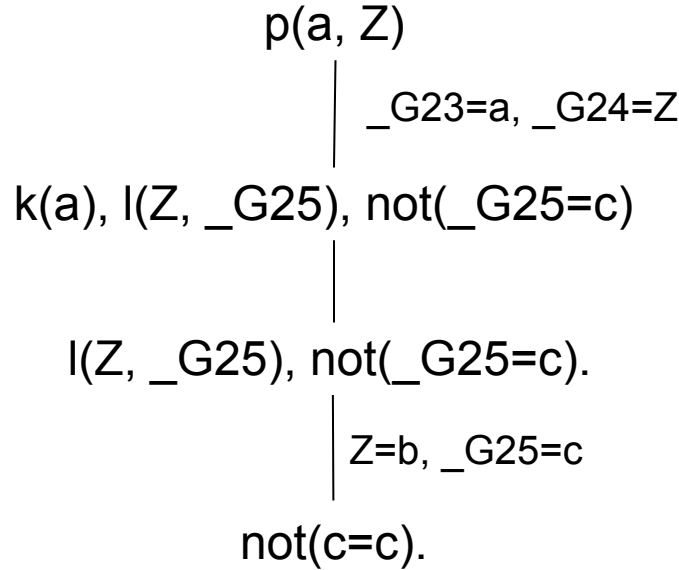
```

k(a).
l(b,c).
l(b,d).
p(X, Y) :- k(X), l(Y, Z), not(Z=c).

?- p(a, Z).
    
```

Match with second fact

Search Tree – Variables



Program

```

k(a).
l(b,c).
l(b,d).
p(X, Y) :- k(X), l(Y, Z), not(Z=c).

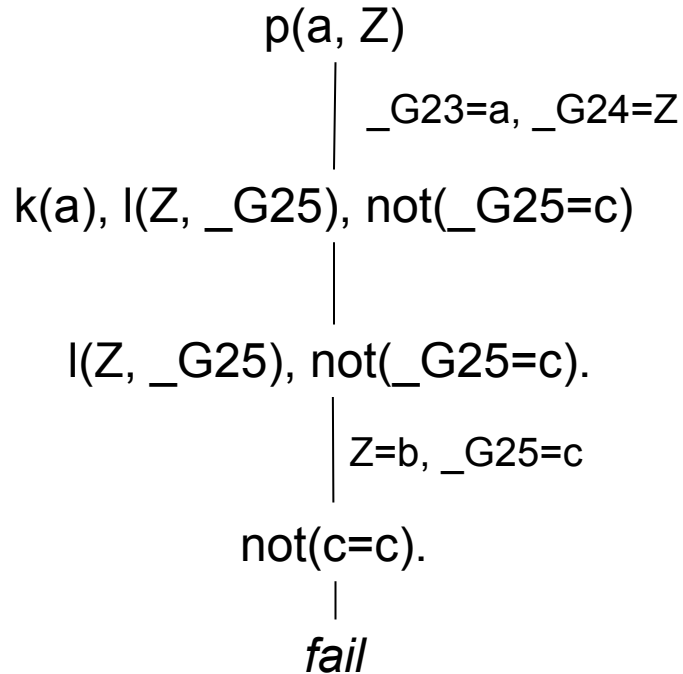
?- p(a, Z).
    
```

Construct finitely failing tree for $c=c$...



$$\begin{array}{c}
 c=c \\
 | \\
 \text{succeed}
 \end{array}$$

Search Tree – Variables



Program

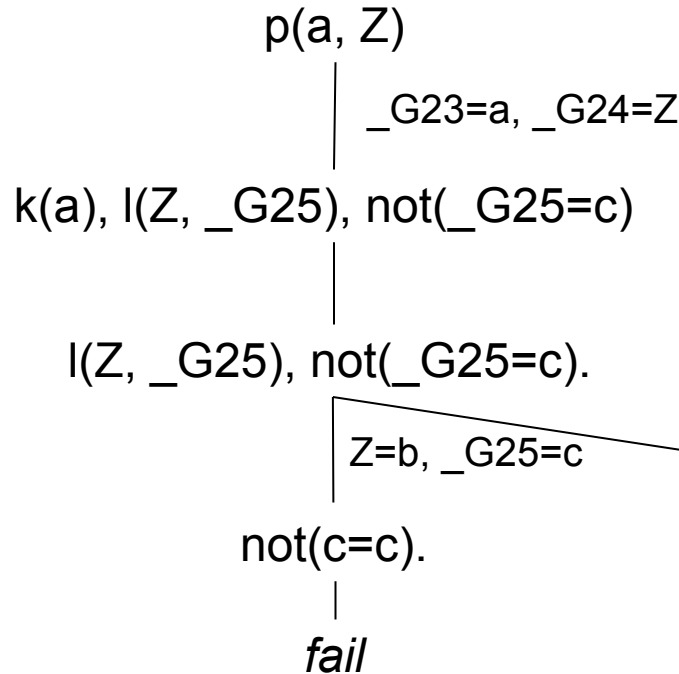
```

k(a).
l(b,c).
l(b,d).
p(X, Y) :- k(X), l(Y, Z), not(Z=c).

?- p(a, Z).
  
```

Cannot construct finitely failing tree for $c=c$!

Search Tree – Variables



Program

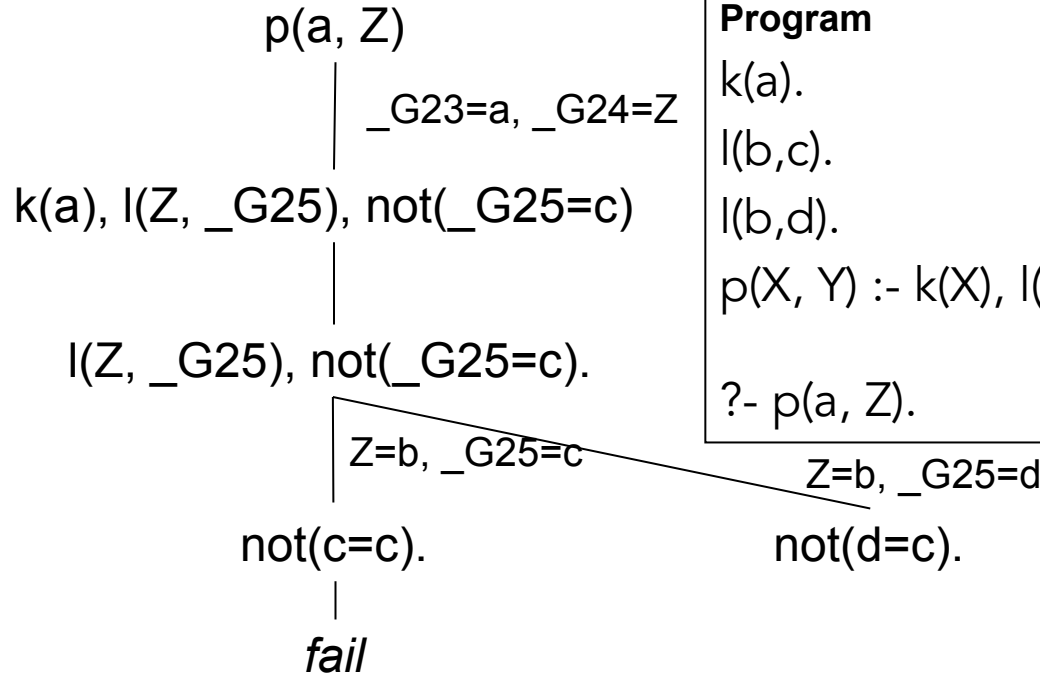
```

k(a).
l(b,c).
l(b,d).
p(X, Y) :- k(X), l(Y, Z), not(Z=c).

?- p(a, Z).
  
```

Backtrack to closest node with alternative choice

Search Tree – Variables



Program

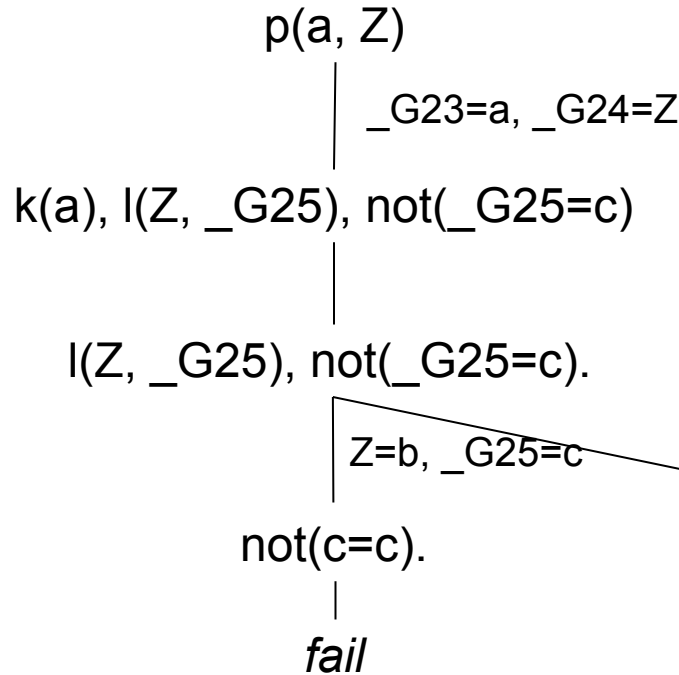
```

k(a).
l(b,c).
l(b,d).
p(X, Y) :- k(X), l(Y, Z), not(Z=c).

?- p(a, Z).
  
```

Match with third fact.

Search Tree – Variables



Program

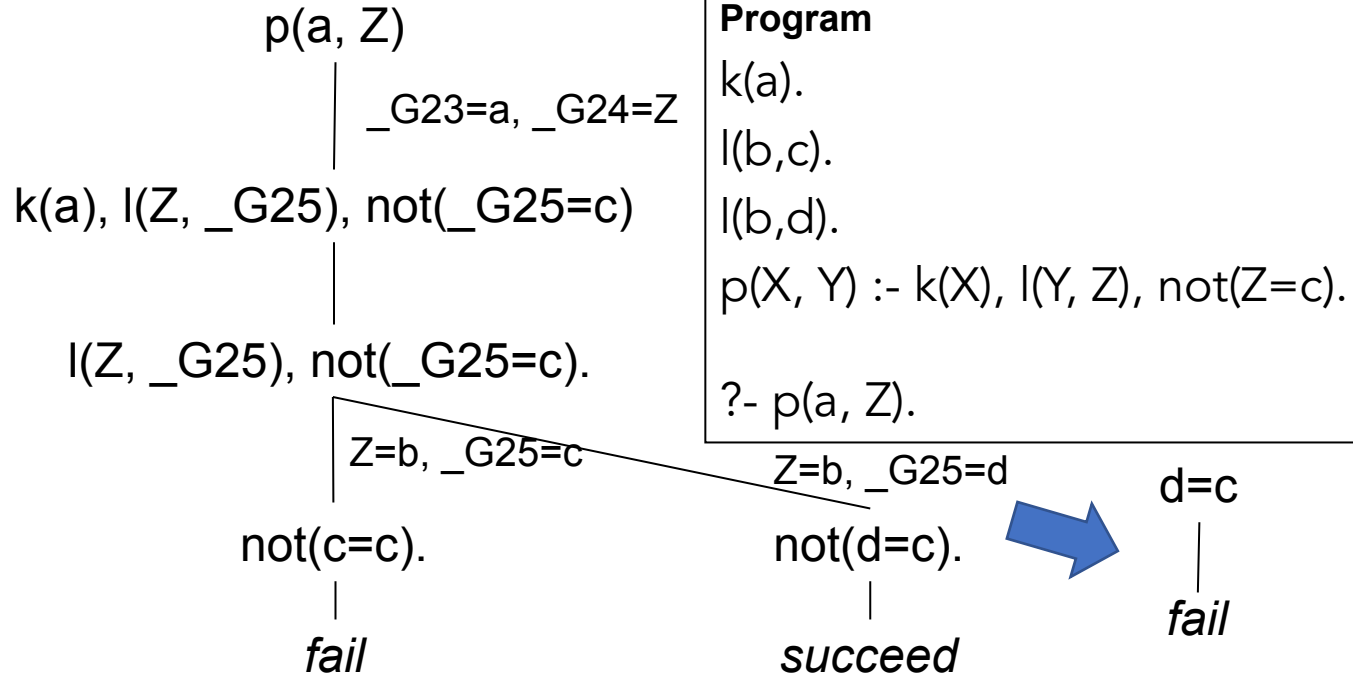
```

k(a).
l(b,c).
l(b,d).
p(X, Y) :- k(X), l(Y, Z), not(Z=c).

?- p(a, Z).
  
```

Construct finitely failing tree for d=c.

Search Tree – Variables



not(d=c) succeeds, done (no choice points left!!).

Search tree example

f(a).
f(b).
g(a).
g(b).
h(b).
k(X) :- f(X), g(X), h(X).

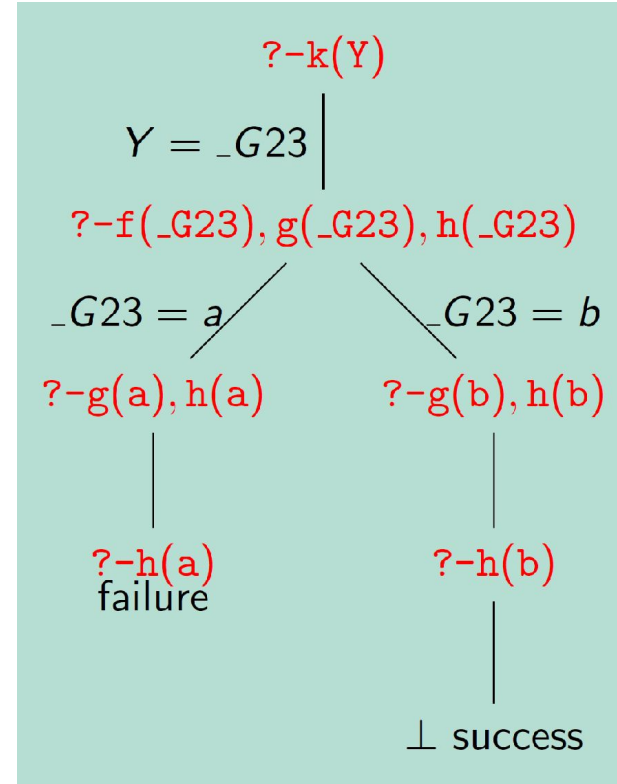
?- k(Y)

Search tree example

```
f(a).
f(b).
g(a).
g(b).
h(b).
k(X) :- f(X), g(X), h(X).
```

```
?- k(Y)
```

Use 'trace' and read the output.



To remember most

How prolog search works:

1. Ask a query
2. Find appropriate rules/facts
3. Prove body of rule
4. Continue until no alternatives remain

Principles of Prolog search:

- 1: Backward chaining
- 2: Linear traversal
- 3: Backtracking

Ends with: Success, Failure or Infinite

Standardization of variables

Learning outcomes

Unification

1. Student can explain how unification works in Prolog using the matching operator `=/2`.
2. Student can derive the result of a query that includes the matching operator.
3. Student can create new queries using the matching operator.

Search

1. Student can explain how search works in Prolog using backward chaining, linear traversal, and backtracking.
2. Student can create a search tree for a given program P and query Q.

Planning:

- Tuesday November 4
 - Lecture on Prolog datatypes, calculation, negation
 - Preparation: Learn Prolog Now Ch. 5.1, 5.2, 10.3
- Thursday November 6
 - Work on prolog assignment 1
 - **No lecture**
 - Preparation: Learn Prolog Now Ch. 3, 4, 5.3, 5.4
- Monday November 10
 - Deadline assignment 1